Mesh Generation and Refinement for Three-Element Airfoil

Suyuan Hu, Takahiro Soeda, and Tongjie Li

February 1, 2023

This project aims to get the meshes of a given geometry. Gmsh was applied to generate coarse meshes. Python and Matlab codes were written and implemented to refine the meshes. As a result, three refined meshes were generated with approximately 8k, 32k, and 128k elements.

1 Introduction

The objective of this project is to generate unstructured, triangular meshes for a given geometry as shown in figure 1. bounded by a far-field square boundary. The geometry given was a three-element airfoil in txt form, which contains the coordinates of the nodes on the airfoils. The given geometry was first used to generate coarse meshes in Gmsh. The meshes generated were then processed to be analyzed and stored for further utilization. After that, Python and Matlab codes were written and implemented to generate local and uniform refined meshes. The expected meshes with approximately 8k, 32k, and 128k elements were generated as a result.

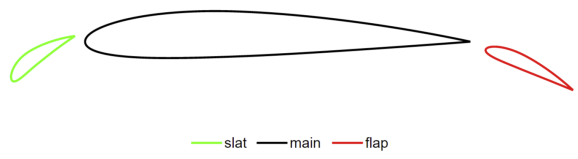

Figure 1: Given geometry

2 Methods

In this project, several refined meshes of the geometry were generated based on the coarse mesh generated with the help of existing software. There were some pre-processing steps and data structure preparations before refining the meshes. Those were based on some general equations shown below.

2.1 Equations

Parameters like the normal vectors of an edge and the area of an element were prepared before refining the meshes. Some base equations were required. Therefore, the main equations for getting those parameters were shown below. The equation for finding the normal vector is as such,

$$L = \sqrt{(P_2[0] - P_1[0])^2 + (P_2[1] - P_1[1])^2} \tag{1}$$

$$N = [\frac{[P_1[1] - P_2[1]}{L}, \frac{P_2[0] - P_1[0]]}{L}], \tag{2}$$

where $P_1[0]$ and $P_1[1]$ represent the x and y coordinates of the first point, respectively, similarly $P_2[0]$ and $P_2[1]$ those of the second point. N is the normal vector calculated. The area of a triangular element can be calculated as,

$$Area = \frac{1}{2} * |x_1 * (y_2 - y_3) + x_2 * (y_3 - y_1) + x_3 * (y_1 - y_2))|, \tag{3}$$

where x_1 and y_1 represent the x and y coordinates of the first point, similar as x_2, y_2, x_3, and y_3 for the second point and third point. The equation used to calculate the maximum magnitude of the error E_e during the mesh verification test is,

$$E_e \equiv \sum_{i=1}^{3} \overrightarrow{n}_{ei}^{outward} l_{ei}, \tag{4}$$

where E_e represents the total error of an element e, $\overrightarrow{n}_{ei}^{outward}$ is the normal vector of the edge i of element e pointing outward, and l_{ei} is the length

of the edge i of element e. In order to smooth the mesh nodes after the refinement, this equation was applied to the nodes to find the new weighted location of the nodes,

$$\vec{x_i}' = (1-\omega)\vec{x_i} + \frac{\omega}{|\mathcal{N}(i)|}\sum_{j\in\mathcal{N}(i)}\vec{x_j}, \tag{5}$$

where $\vec{x_i}'$ is the new location of the node omega is the weight for the adjacent nodes in the calculation of the new coordinate, $\mathcal{N}(i)$ is the set of the nodes adjacent to the node i, $\vec{x_i}$ is the coordinate of the current location of the node, and $\vec{x_j}$ is the location of the neighbors.

2.2 Algorithms and Code

Algorithms were split into several parts according to their functions.

2.2.1 Pre-processing

The algorithm of the codes to process the mesh for matrices `I2E`, `B2E`, `In`, `Bn`, and `Area` were built like the following.

Like the given sample code `egdehash.m`, the determination of interior and boundary edges were based on the time of appearance of an edge in a for-loop. This way of building the code would save running time but would give more burden on the storage. At the beginning of the code for `I2E` and `B2E`, a sparse matrix with the size of the amounts of nodes was created to store the edge information. Each number in the matrix either represents the local face number in an element (1,2, or 3) or is -1. The column and the row number represent the nodes on the two sides of the edge. Two for-loops, one for elements and the other for each edge in an element, were used for looping over all the edges. As shown in figure 2, if an edge has not been flagged, the corresponding HH number would be 0 and pass the if statement and assumed to be the boundary face and marked with its face number. If a value of HH was greater than zero, it means that it's the second time the corresponding edge shows up, therefore it is an edge shared by two elements. In this case, this edge would be considered as an interior edge and pass the second for-loop. The information of this edge would then be recorded in the `I2E` matrix. The corresponding value of HH would be marked as -1 so that the boundary edges could be simply selected by filtering out every negative value of HH.

```
if (H(n1,n2) == 0)
  HH(n1,n2) = oppositenode;  HH(n2,n1) = oppositenode;
else
  oldoppositenode = HH(n1,n2);
  if (oldelem < 0), error 'Mesh input error'; end
  niedge = niedge+1;
  I2E(niedge,:) = [oldelem, oldoppositenode, elem, oppositenode];
  HH(n1,n2) = -1;  HH(n2,n1) = -1;
```

Figure 2: Interior and boundary edge determination

To make it consistent that all the elements on the left are the ones with smaller numbers, the edge information was also stored on the opposite side of the matrix. So simply taking the upper side of the diagonal of a matrix would be sufficient to accomplish that.

When calculating the normal vectors for interior and boundary edges, the coordinate information was stored for the two nodes on the two sides of each edge. Then with the coordinates, the interior and the boundary normal vectors can be computed by implementing equation 2. Similarly, with storing the three nodes in an element, the area of an element can be calculated by employing equation 3.

2.2.2 Verification Test

In the verification process, four lists were required as input: [Inner edges], [Boundary edges], [Inner edge normal], [Boundary edge normal]. Several functions were pre-set to serve this algorithm for the verification:

[`length`]: Took two points' coordinate as input and output the distance between.

[`norm`]: Took two points' coordinate as input and output the norm vector for P1 to P2

[`get area`]: Took three points' coordinate as input and output the triangle's area formed by these three points.

Inner edges and boundary edges lists were used to get the length of these edges using the [length] function. The edge normal was stored in edge nor-

mal lists in the same sequence as the edges stored. The verification was calculated by the equation 4. In the process, the results of each edge multiplying its normal were added to a new verification zero list which had the same size as the list storing "element to nodes". After all the edges including boundary ones and inner ones were processed. The expected outcome list should have the machine precise zero in every index (less than 10e-15).

2.2.3 Refinement

In this part, several functions were pre-set to serve this algorithm. [tri central]: Took three points' coordinates as input and output the triangle's geometry center formed by these three points. [mid point]: Took two points' coordinates as input and output coordinates of their midpoints. [flag edges]: Took an element to node format (A, B, C) as input and output the three new middle nodes' coordinates on each edge by using the [mid point] function. [ch angle]: Took three points' coordinate as input and output the formed triangle's largest angle between $\angle P3P1P2$ (output as 1) and $\angle P3P2P1$ (output as 2).

In the refinement process, a circle of refinement area with center (x, y) and radius r was chosen first.

Write function [refine zone]: [refine zone] took five inputs: list for element center, list for element to nodes, list for node coordinates, refine center point P and refine radius r. A [refined] list was created to store the element that had been refined. Node number counter "n" was created to be equal to the size of the input list for node coordinates.

Every element center (gained by using the [tri center] function) in the mesh was looped over to find which ones have less distance to the refinement center than radius r using the [length] function. In the chosen element, function [flag edges] was used to pick up the three new nodes created in this element and then added to the node coordinate list. During the adding process, newly created nodes could be found to be already exist in the previously refined elements if they are adjacent to each other. So the new nodes need to be not in the previous node coordinate list to be added.

A list p was created to store the node number for each newly created node. If a newly created node was not in the node coordinate list before, it would be added to the list and get the node number n + 1. The counter n would increase by 1 at the same time. If a node was already in the node coordinate list, it would not be added and the node number would be searched in the list to be indexed + 1. The new nodes were processed by the sequence of P1, new P1, P2, new P2, P3, new P3 and their node number added to the list p

Four new elements were created using the new node number created to insure each was in the counterclockwise direction, NE1 = [p[0], p[1], p[2]]; NE2 = [p[0], P2,p[1]]; NE3 = [p[1], P3,p[2]]; NE4 = [p[2], P1,p[0]]. Four new elements were added to the old element in the node list. The refined element was added to the refined list. The new node coordinate list, the element to node list, and the refined list were the outputs.

The function [refine zone] only refined the elements with its center located in the refinement zone, the following function helped refine the elements that were not in the zone but adjacent to elements being refined before.

Write function [refine2]: [refine2] took three inputs: a list for refined elements, a list for inner nodes to edges (I2E), list for node coordinates. A [flag] list was created to store the information about which edges in one element had been flagged.

Listing 1: Codes flagging the refined edges

```
for i in range(len(refined)):
    for j in range(len(I2E)):
        if I2E[j][0] == refined[i] and I2E[j][2] not in refined:
            flag[I2E[j][2]-1][I2E[j][3]-1] = 1
        elif I2E[j][2] == refined[i] and I2E[j][0] not in refined:
            flag[I2E[j][0]-1][I2E[j][1]-1] = 1
```

By using the code shown above, the edges that were flagged were stored as one, and others were kept as 0.

By looping over the list [flag], the elements that have one or two edges flagged were found. The index of each line in the list [flag] is the edge opposite to the node that has the same sequence in the element-to-node list. By adding up each line in the list [flag], the elements that had a result of 1 indicate that it had one edge been flagged.

The flagged edge should be the edge with index 1 in the [flag] list. The node number of the newly added node on this edge was found out by searching its coordinate in the node coordinate list. With the new node, two new elements were created and added to the existing element-to-node list.

The elements that had a result of 2 indicate that it had two edges flagged. Two newly created node and their node number could be found by using the method same as before. The difference was to determine which elements need to be created by judging the angles that were larger (The larger angle got spilt into two angles in two elements.) Using the [ch angle] function, the larger angle was found, and three new elements were created and added to the existing element-to-node list. Each step above was followed by adding the refined element number to the existing [refined] list. The new node coordinate list, the element-to-node list, and the refined list were outputted.

Write function [smooth]: [smooth] took 3 inputs: smooth center point P, smooth radius r, node coordinate list Boundary edge lists were output again with the mesh result after the refinement. By looping over all the indices in the boundary edge list, we get all the nodes on the boundary in a list [Bnode]. List [refined N] created to store the node needs to be smoothed. All the node coordinates were looped over to find out the nodes locates within the smoothing circle while also not in the boundary node list. Adding those nodes to the nodes' number to [refined N] list.

For each node in [refined N], a list [nearN] was created to store all the nodes adjacent to it with the first index of the node itself. Looping over the element to node list and finding all the nodes share the same element with it. Those nodes were then added without repeat to the [nearN] list.

The new coordinate for the node was calculated by the equation 5

The node coordinate was changed in the node coordinate list for this node. The new node coordinate list was outputted after all nodes in list [refined N] were smoothed.

3 Results

In this section, the results of the tasks of this project were presented.

3.1 Task 1

In the very first step, the files `slat.txt`, `main.txt`, and `flap.txt` were converted into files in format that were readable for Gmsh. The python code for doing that was called *`txt2geo.py`*. In Gmsh, four points with coordinates of [−100, −100], [−100, 100], [100, 100], [100, −100] were generated to create the far-field boundary. The 2D mesh general settings were set as shown in figure 3.

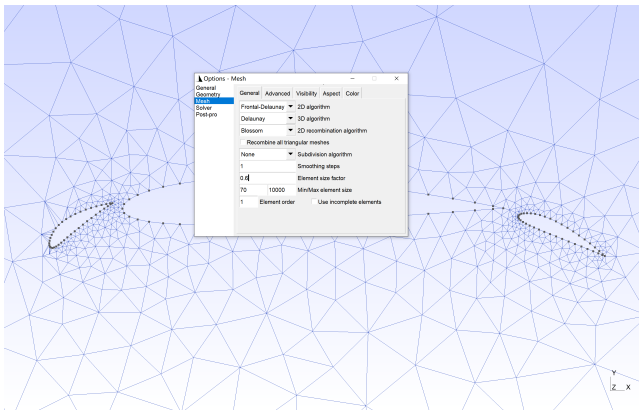

Figure 3: Gmsh general mesh setting

Then, a coarse, unstructured, triangular mesh with 1288 elements was generated as shown in figure 4 by choosing "2D mesh" in Gmsh.
The mesh generated was then saved in `msh` format called `all.msh`. To further refine and plot the meshes with Matlab and Python codes, the `msh` file was converted into `gri` format, which was an in-house mesh file format. This file contains the node coordinates, a list of boundary groups and faces, and a list of elements.

3.2 Task 2

In this section, codes were written to process the mesh generated in the previous task. The first two parts of the code were to generate a mapping from interior faces to the elements and from boundary faces to the elements. The method used was de-

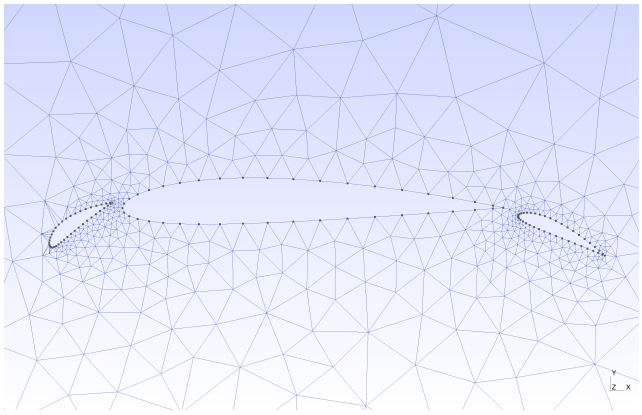

Figure 4: Coarse, unstructured, triangular mesh with 1288 elements in Gmsh

scribed in section 2. Test mesh was run to examine the correctness of these codes. The output matrices were printed out and shown in figure 5.

```
I2E = 1   1   2   2
B2E =
      1       3       1
      1       2       1
      2       3       1
      2       1       1
```

Figure 5: I2E and B2E matrices for the test mesh

Then, the normal vectors for the interior and boundary faces were found along with the area of the elements. The methods used in these steps were demonstrated in the second section too. The corresponding matrices for the test meshes were shown in figure 6.

```
In =  [[0.70710678 0.70710678]]
Bn =
 [[-1.  0.]
 [ 0. -1.]
 [ 0.  1.]
 [ 1.  0.]]
Area =  [0.5, 0.5]
```

Figure 6: In, Bn, Area matrices for the test mesh

3.3 Task 3

In the verification process, the result got by using equation 4 was stored in a list. A part of it could be seen in figure 7. All the index in the list was close to zero, to machine precision.

0	0	0
1	0	0
2	0	-6.93889e-18
3	0	0
4	0	0
5	0	0
6	0	0
7	3.46945e-18	3.46945e-18

Figure 7: List for the verification result

3.4 Task 4

In this section, the generated mesh was refined. Four regions: slat leading edge, gap between the slat and main airfoil, gap between main airfoil and flap, flap trailing edge were refined. The mesh before the refinement was shown by figure 8. After smooth and spline, the mesh result was shown by figure 9. The mesh node number grew from 712 to 1093. The total element number grew from 1288 to 2004.

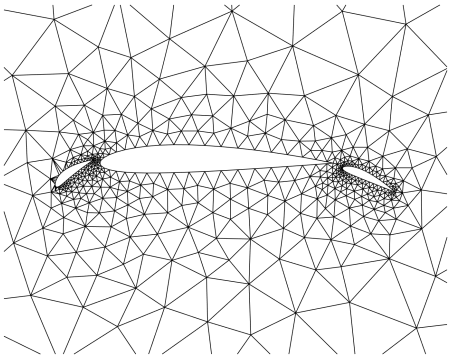

Figure 8: Affected mesh regions before the refinement

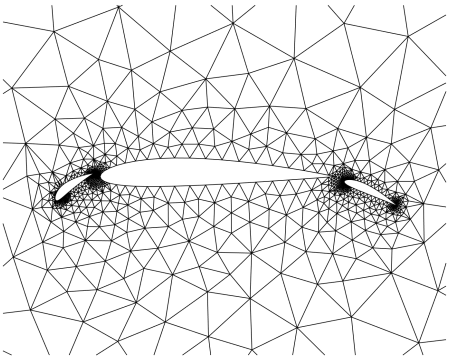

Figure 9: Affected mesh regions after the refinement

3.5 Task 5

In this section, The uniform meshes were applied to the previous refinement result. Each uniform mesh quadrupled the total element number.

Figure 10 showed the result of one uniform mesh (8k element).

Figure 11 ,figure 12, figure 13 showed the result of two uniform meshes (32k element).

Figure 14, figure 15, figure 16 showed the result of three uniform meshes (128k element).

All of the meshes pass the verification test with the generated [ln] Inner edge normal lists and [Bn] boundary edge normal lists.

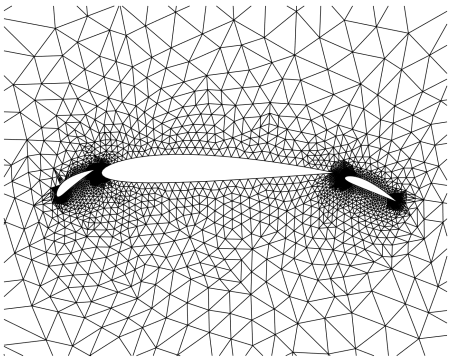

Figure 10: Meshes with uniform refinement with approximately 8k elements

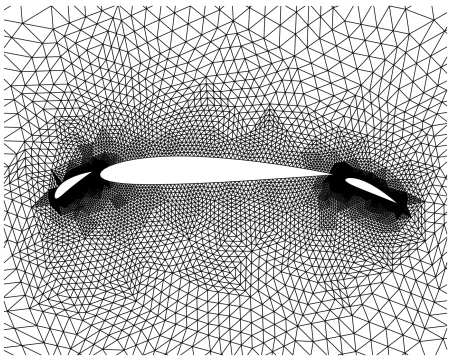

Figure 11: Meshes with uniform refinement with approximately 32k elements

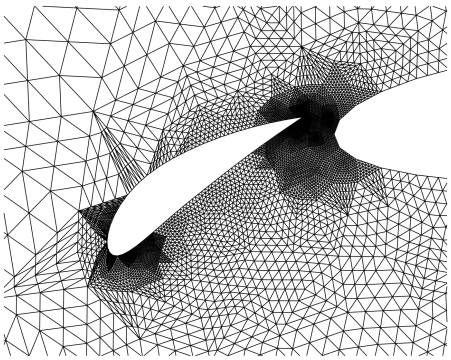

Figure 12: Front Meshes with uniform refinement with approximately 32k elements

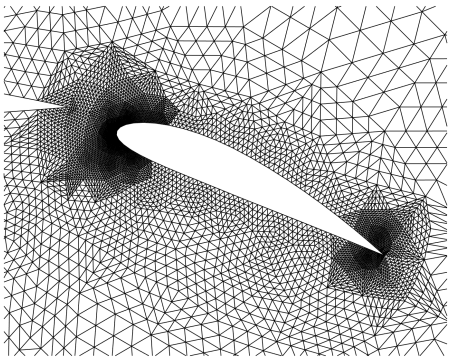

Figure 13: Back Meshes with uniform refinement with approximately 32k elements

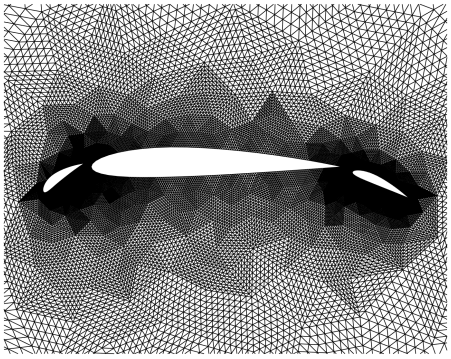

Figure 14: Meshes with uniform refinement with approximately 128k elements

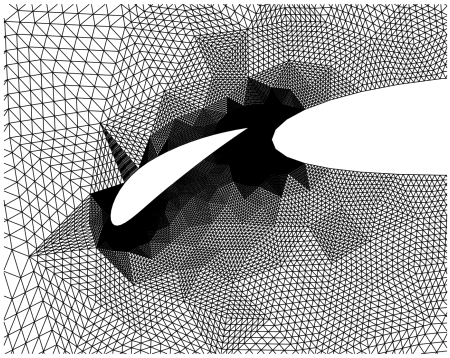

Figure 15: Front Meshes with uniform refinement with approximately 128k elements

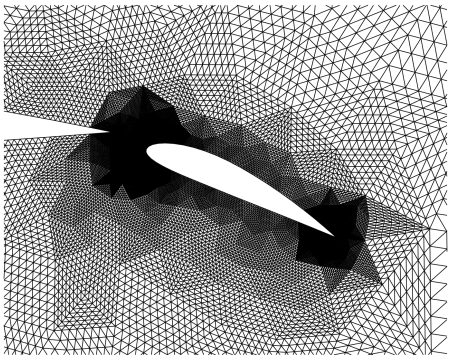

Figure 16: Back Meshes with uniform refinement with approximately 128k elements

4 Conclusions

In all, three different refinement level meshes were generated for the given geometry. Four locations were chosen to be locally refined due to their ability to determine the quality of the results. Better refined meshes mean a larger number of elements, which is also more time-consuming in generating the mesh. Therefore, a good judgment on mesh selections needs to be made before meshing.

5 Effort Breakdown

Suyuan, Takahiro, and Tongjie finished the first task together. Suyuan found the interior and boundary faces in task 2 and did the verification test in task 3. Takahiro found the area of the elements. Tongjie computed the matrices of I2E and B2E in task 2 and finished task 4 and task 5 with Suyuan. Suyuan and Tongjie wrote the report together. Tongjie put all the codes in order.

Table 1: Effort percentages.

	S. Hu	T. Soeda	T. Li
development	33	33	33
coding	30	50	20
managing	40	20	40
report	33	33	33